

Guía de Trabajo con Git & GitHub

Flujo de Desarrollo Basado en Ramas (Feature Branch Workflow) y CI

Esta guía establece el flujo de trabajo oficial para el desarrollo del proyecto. El objetivo es mantener el código de la rama principal (`main`) siempre estable, funcional y protegido contra errores individuales, apoyándonos en la Integración Continua (CI).

1. Configuración Inicial (Paso 0)

Antes de comenzar a trabajar por primera vez, es fundamental asegurarse de tener una copia limpia y bien vinculada del repositorio remoto en tu máquina local.

Clonación del Repositorio

Para evitar conflictos de carpetas y asegurar que el nombre local coincida exactamente con el de GitHub, utiliza el siguiente comando en la terminal dentro de tu directorio de proyectos:

```
git clone https://github.com/USUARIO/NOMBRE-REPOSITORIO.git
```

Nota: Sustituye la URL por la dirección real de nuestro repositorio. Esto creará automáticamente una carpeta con el nombre exacto del proyecto y la vinculación correcta al origen remoto.

2. Ciclo de Desarrollo: Creación de Características

Está **estrictamente prohibido** programar o realizar cambios directamente sobre la rama `main` . Cada nueva funcionalidad, botón o corrección debe desarrollarse en una rama propia.

A. Preparación de la Rama (Cada vez que empieces algo nuevo)

Antes de escribir una sola línea de código, debes sincronizar tu entorno local con los últimos cambios del equipo y abrir una rama limpia:

1. Regresa a la rama principal:

```
git checkout main
```

2. Descarga e integra las últimas actualizaciones del servidor:

```
git pull origin main
```

3. **[Si aplica]** Elimina la rama local de la tarea anterior ya finalizada (para mantener limpio tu entorno):

```
git branch -d nombre-rama-anterior
```

4. Crea y muévete a una nueva rama descriptiva:

```
git checkout -b nombre/tarea-especifica
```

💡 Regla de Oro para el Nombre de las Ramas

En lugar de usar siempre el mismo nombre, utiliza nombres dinámicos y descriptivos estructurados como: `nombre/breve-descripcion`. Ejemplos: `sofia/boton-login`, `jose/conexion-base-datos` o `patri/ajuste-estilos`. Esto facilita el rastreo de cambios en GitHub.

B. Buenas Prácticas al Programar

- **Funcionalidad Mínima:** Sube código que sea "medianamente funcional" (por ejemplo, una vista maquetada, un botón que ya responde o un método del backend con sus pruebas). Evita hacer *push* por cada línea escrita, pero tampoco acumules semanas de trabajo sin subir.
- **Confirmaciones Locales (Commits):** Puedes hacer tantos commits locales como desees mientras avanzas en tu máquina (`git commit -m "Mensaje descriptivo"`).

3. Publicación de Cambios y Revisión (Pull Request)

Una vez que la funcionalidad está lista y verificada en tu entorno local, llega el momento de compartirla con el equipo y fusionarla con `main`.

Paso 1: Subir la rama al servidor remoto

```
git push origin nombre/tarea-especifica
```

Paso 2: Apertura de la Pull Request (PR)

1. Accede a la interfaz web de nuestro repositorio en GitHub.
2. Aparecerá automáticamente un aviso destacado de color amarillo con el botón "**Compare & pull request**". Haz clic en él.
3. Redacta un breve título y una descripción de lo que has implementado (ej. "*Añadido el controlador de autenticación y validación de formularios*") para que el resto del equipo entienda el cambio.

Paso 3: Validación por Integración Continua (CI)

Al abrir la Pull Request, se activará de forma automatizada la canalización de **Integración Continua (CI)**. Deberás esperar en esa misma pantalla (aproximadamente 1 minuto) a que el sistema compile el proyecto y ejecute las pruebas de verificación.

- **Check en Verde (✓ Pasado):** Indica que el código no rompe la estructura del proyecto ni invalida los tests existentes. El botón "**Merge pull request**" se habilitará.
- **Cruz en Rojo (✗ Fallido):** Significa que el nuevo código impide la compilación o rompe alguna funcionalidad existente. GitHub **bloqueará automáticamente** la fusión para proteger al resto del equipo.

⚠️ ¿Qué hacer si la CI falla o hay Conflictos?

Si la CI se pone roja o GitHub avisa de que hay "Merge Conflicts" (conflictos de fusión), significa que otra persona modificó las mismas líneas de código que tú en `main`. Para solucionarlo **no vuelvas a empezar de cero**. Haz lo siguiente en tu terminal:

```
git checkout main
git pull origin main
git checkout tu-rama-con-problemas
git merge main
```

Resuelve los conflictos en tu editor de código (IDE), realiza un nuevo commit con la solución y vuelve a ejecutar `git push origin tu-rama-con-problemas`. La Pull Request en la web se actualizará sola.

Paso 4: Fusión (Merge)

Si todo está en verde, pulsa el botón **"Merge pull request"** y posteriormente **"Confirm merge"**. Tu código ya formará parte oficial de `main`. ¡Listo! Para la siguiente tarea, vuelve al inicio de la sección 2.